

The Compile Worker Process

A plain-language account of source-bounded atomic extraction, its controls, and its evidence

The Worker is a model-assisted extraction engine inside a deterministic control shell. It does not decide that source material is correct or canonical. It preserves potentially relevant claims as small, traceable evidence records for later human triage and reconciliation.

Atomic extraction phase - MEDIAN v0.5.0

Where this fits in the compile

The compile is the complete controlled pipeline:

- Atomic extraction - the current phase.
- Human and deterministic triage.
- Semantic mapping and comparison.
- Grand Reconciliation.
- Final specification construction and verification.

The Worker is currently authorized only for atomic extraction. An accepted extraction candidate is evidence prepared for later judgment, not accepted game canon.

1. Cold start

Whenever the Worker begins or resumes, it reads the repository operating contract and canonical state rather than relying on thread memory. It confirms the active phase, completed and next sources, chunk boundary, authorities, remaining provider budget, prohibited stages, STATUS parity, Git synchronization, and the full guard.

Canonical state is the single machine-readable JSON file that governs current operations. It is authoritative.

STATUS.md is only a derived human-readable dashboard.

2. Sparkup and source selection

A normal instruction to resume or proceed grants work on exactly one source. The Worker derives that source from canonical compile state and the processing order; it does not select by memory or filename intuition. Source-work authority ends at source completion.

Sparkup is the informal name for returning source-work authority to the Worker. It is a lifecycle concept, not a required magic phrase.

3. Identity investigation

Before extraction, the Worker binds the exact source ID, path, SHA-256, role, evidence stream, source-declared status, media treatment, exclusions, and extraction boundary in an identity card.

SHA-256 is a content fingerprint. Even a one-character edit changes it, preventing the engine from preparing against one document and unknowingly processing another.

Identity approval confirms the extraction boundary - which exact document to process and how - not the truth or canonical status of its individual claims. The card remains minimal because it is provider-visible.

4. Offline preparation

The source is parsed into stable **blocks**: paragraphs, list items, table rows, headings, captions, or source-position markers. Each receives a stable block ID. Blocks are assigned roles:

- Target block - Anthropic must explicitly decide what extractable claims it contains.
- Context block - visible for interpretation, but receives no returned disposition.
- Excluded block - removed offline because it is known non-content.
- Structural target - explicitly accounted for as containing no substantive claim.

A **disposition** is the accounting decision for one target: **atoms**, **no_substantive_claim**, or **review_required**. No result must be explicit; invisible omission is not acceptable.

5. Chunk planning

A **chunk** is a bounded package of target blocks plus necessary context. It is not an arbitrary token slice. Planning preserves indivisible structures such as a lead-in with its list, a table heading with its rows, or a rule with its qualifications.

Target density is the calibrated number of provider-eligible targets per chunk. The chunk count is generated from this density while accounting for the entire source.

6. Prompt, schema, and configuration

- Prompt - semantic instructions for extracting claims faithfully.
- Response schema - the exact permitted JSON structure, IDs, fields, and values.
- Machine configuration - the binding among source, identity, chunk plan, prompt, schema, model, cache, and cost rules.

The prompt guides meaning; the schema restricts structure; the validator verifies the returned result.

7. Fake validation

Fake validation makes no provider call. Synthetic valid and defective responses are run through the real validators. This proves that the machinery can represent and accept a complete answer, and reject known defects for the expected reason. It tests the machine, not Anthropic's intelligence or the quality of future atoms.

8. Representative pilot calibration

Before releasing a whole source, the Worker chooses a representative chunk that exercises its characteristic difficulty. One real call tests target density, reasoning level, output allowance, prompt clarity, schema strictness, and validation behavior.

Calibration may correct or simplify the existing source-agnostic machinery. It may not create a special extraction system merely because one source is difficult.

9. Freezing a call packet

A **call packet** binds the source and chunk IDs, targets, context, exclusions, prompt, response schema, identity card, model, reasoning, cache policy, required disposition count, hashes, and conservative cost ceiling.

Frozen means those ingredients are fixed and hash-bound for that attempt. If a binding changes, the old attempt remains preserved and a new packet version is frozen.

10. Preflight

Preflight is the final deterministic check before transmission. It confirms active authority, standing Anthropic consent, hashes, sequential order, prior review, sufficient budget, credential availability, and valid cache and cost settings. It makes no provider call.

The **conservative cache-miss ceiling** is a cautious maximum reservation. The budget must cover it, although the actual charge is normally lower. Reservation is not spending; exact usage replaces the estimate after the call.

11. Provider call and caching

The frozen packet is transmitted to Anthropic. A stable prompt prefix is marked for one-hour caching. The first eligible call creates the cache; later calls can read it more cheaply. Cache telemetry distinguishes creation, reads, ordinary input, output, and reasoning tokens.

Transport means the actual movement of the request and response. A transport failure differs from a bad response because Anthropic may have received nothing and no spend may have occurred.

12. Raw response and compact outcome

The **raw response** preserves exactly what Anthropic returned. The **compact outcome** records HTTP status, usage, exact cost, caching, stop reason, structured proposal, validation findings, and review state. The compact form reduces repeated rereading while the raw evidence remains available.

13. Mechanical validation

Programmatic checks cover:

- Valid JSON and schema compliance.
- Exactly one disposition for every target block.
- No missing, duplicate, invented, or context-only IDs.
- Required disposition and evidence-stream rules.
- Exact quoted evidence occurs in the claimed source block.
- Cardinality, table-cell coverage, and detectable atomicity or self-containment defects.

Grounding means an atom's quoted evidence is a contiguous, exact source span. **Coverage** means every target and independently meaningful assertion was accounted for. **Atomicity** means one atom expresses one independently reusable claim.

14. Substantive review

The Compile Worker compares proposed atoms with the source for omissions, invented implications, weakened or strengthened language, lost conditions or negatives, changed subjects, examples turned into universal rules, and historical or provisional claims presented as current.

The **normalized claim** is a concise statement that can be understood independently while preserving the force of the source. The exact source text remains beside it. **Claim kind** is a functional label such as rule, definition, example, procedure, constraint, or qualification. A **stream** routes the evidence type; it is not another phase.

15. Acceptance, rejection, and retry

A chunk is accepted only after mechanical and substantive review pass. Here, accepted means that the chunk was faithfully extracted - not that every statement is correct or canonical.

A failed attempt and its cost remain preserved. Ordinary defects may be corrected through the smallest source-agnostic prompt, schema, validator, chunking, or engine repair, followed by offline tests, replay, refreezing, and retry. Unusual interpretive or structural questions halt for review.

Replay runs preserved responses or fixtures through revised logic. **All-source compatibility** checks that a generic repair does not fix one source by breaking others.

16. What the Worker means by a test

The word test is overloaded and may refer to:

- Unit test - one piece of Python behavior.
- Regression test - a previously corrected behavior that must remain corrected.
- Negative test - deliberately invalid material that must be rejected.
- Fake validation - synthetic provider-shaped output processed offline.
- Replay test - preserved evidence processed under revised validation logic.
- Compatibility test - generic behavior checked across source fixtures.
- Pilot - one real provider call used for source calibration.
- Mechanical validation - deterministic checks on one real response.
- Substantive review - semantic comparison by the Worker.
- Full guard - integrated contract, state, artifact, and automated-test validation.

Thus, '82 tests passed' does not mean 82 provider calls occurred.

17. Ledger and spend accounting

A source **run ledger** records attempts sequentially, usually as one JSON object per line. Entries are hash-linked so later substitution breaks the chain. Canonical state holds current spend authority; the ledger supplies detailed history rather than competing authority.

The active budget is a **spend envelope**. While source authority is active and its balance covers the next conservative ceiling, sequential calls need no transaction-by-transaction approval.

18. Whole-source acceptance

After the final chunk, the Worker verifies complete chunk coverage, excludes rejected attempts, checks traceability and ordering, prevents media or alternate-source leakage, and performs whole-document extraction-quality review.

The resulting **accepted candidate** is a JSONL collection of traceable atoms plus an acceptance report. Contradictory, obsolete, provisional, or superseded statements can all be correctly extracted candidates for later triage and Grand Reconciliation.

19. Checkpoint, halt, and Stopdown

A **checkpoint** is a coherent commit and push that protects progress. It does not necessarily relinquish authority. A **halt** is any pause - spend, approval, ambiguity, transport, structure, or completion - and is not automatically a Stopdown.

Formal **Stopdown** requires:

- Source-work and repository-write authority both set false.
- Candidate packaging completed where applicable.
- Canonical state and the deterministic dashboard refreshed.
- Full guard passed.
- Closing checkpoint committed and pushed.
- Clean worktree with local HEAD equal to origin/main.

Only then does the Worker notify the Supervisor that it has Stopped Down.

20. STATUS.md and the full guard

STATUS.md is generated from canonical state before full-guard and publication checkpoints. The model does not compose it as live prose, so it cannot become parallel authority or drift from machine state.

The **full guard** is the integrated safety check for the operating contract, canonical state, source bindings, hashes, authorities, dashboard freshness, evidence structures, and automated tests.

The process in twelve steps

- Prove which exact document is being handled.
- Divide it into accountable source blocks.
- Plan bounded chunks without breaking semantic structures.
- Test the machinery offline.
- Calibrate one representative real chunk.
- Send chunks sequentially to Anthropic.
- Preserve every request, response, cost, and failure.
- Mechanically validate every result.
- Substantively review every proposed atom.
- Correct ordinary generic defects without redesigning the pipeline.
- Package a whole-source accepted extraction candidate.
- Relinquish authority through formal Stopdown.

The Worker determines whether the source was faithfully extracted. It does not determine whether the source was right.